

Verifiable Federated Unlearning for Backdoor Removal

BAERASER-Inspired Trigger Recovery as Post-Unlearning Verification

Core ordering: Federated training $\rightarrow$ CrowdGuard detection $\rightarrow$ Wu historical unlearning $\rightarrow$ Wu KD repair $\rightarrow$ BAERASER-Inspired trigger recovery $\rightarrow$ empirical verification $\rightarrow$ commit–reveal.

1. Research Objective

The system does not claim to formally prove that every possible backdoor has been removed. It performs post-unlearning, model-preserving empirical verification by attempting to recover residual trigger behavior without being given the ground-truth trigger.

The four components have distinct roles: CrowdGuard identifies suspicious clients; Wu et al. removes the estimated malicious clients' historical contributions and performs KD repair; a simplified BAERASER-inspired trigger-recovery procedure searches for residual trigger behavior under a stated 6×6 spatial prior; and commit–reveal protects the integrity and auditability of verifier votes.

The verification procedure is empirical and bounded by its recovery search space, optimization procedure, verification data, and thresholds. A failure to recover a trigger is therefore reported only as "no recoverable trigger under the specified protocol," not as proof that the model is backdoor-free.

2. Threat Model and Information Boundary

Verifier knows	Verifier does not know
Post-unlearning model θ_U	Trigger spatial position (u, v)
Limited clean validation data DV	Trigger RGB pixel pattern
Label space L	Attacker target label
Trigger spatial extent = 6×6 (project-specific defender prior; known to verifier)	Poisoned training data

The 6×6 prior is a project-specific defender assumption and is not a requirement of the original BAERASER method. The verifier is assumed to know only the trigger's spatial extent, while the trigger position, RGB pattern, and target label remain unknown. The original BAERASER recovery searches over candidate labels and recovers an unknown, unconstrained trigger distribution with no spatial mask.

The 6×6 size, along with the fixed-corner placement seen in the public reference implementation (openfl-tutorials/experimental/workflow/CrowdGuard/cifar10_crowdguard.py), comes from that repository's demo backdoor injection code rather than from the CrowdGuard paper's text. This project keeps the 6×6 size but explicitly departs from the repo's fixed-corner placement — see Section 8.

3. End-to-End Architecture

CIFAR-10 federated training $\rightarrow$ 6×6 pixel backdoor injection (one randomized position and RGB pattern per attack run) $\rightarrow$ backdoored global model θ_K $\rightarrow$ CrowdGuard HLBIM + stacked clustering $\rightarrow$ estimated malicious-client set M $\rightarrow$ Wu historical-update subtraction $\rightarrow$ Wu knowledge-distillation repair $\rightarrow$ post-unlearning model θ_U $\rightarrow$ independent verifiers run simplified BAERASER-inspired trigger recovery (6×6 extent known; position, pattern, and target label unknown) $\rightarrow$ recovered candidate trigger(s) $\rightarrow$ local

recovered-trigger ASR + clean accuracy → comparison against the same recovery procedure run on a clean-model recovery baseline → empirical verification status → commit-reveal → independent tally.

Critical ordering: unlearn first → recover/test afterward → compare against the predefined recovery baseline → vote last. There is no pre-unlearning recovery decision in the main pipeline.



Figure 1. End-to-end pipeline, left to right: training/detection/unlearning → post-unlearning recovery and verification → vote integrity.

4. Step 0 — Federated Training and Backdoor Injection

Parameter	Main project setting	Qualification
Dataset	CIFAR-10	32×32 images; 50,000 training samples
Logical clients	20	Follows CrowdGuard paper's Table IV default; the public reference implementation's demo uses only 4 clients
Aggregation	Equal-weight FedAvg	1/N weighting, per CrowdGuard
Samples/client	2,500	Chosen for a disjoint 50,000-sample partition across 20 clients
Local epochs	10	Project configuration
Batch size	64	Project configuration
Learning rate	0.01	Project configuration
Attack	Adaptive constrain-and-scale	Per CrowdGuard's adaptive-adversary framing; the sampled 6×6 RGB pattern is the trigger used by the attack, and the adaptive constrain-and-scale procedure governs the poisoned updates; poisoning rate within malicious clients must be fixed and reported for reproducibility
Trigger size	6×6 RGB, fixed	Project-specific prior; traced to the CrowdGuard reference repo's demo, not its paper
Trigger position	One position sampled per attack run; hidden from verifier	Deliberate departure from the reference repo, which hardcodes the trigger at image[:, 0:6, 0:6]
Trigger RGB pattern	One continuous 6×6 RGB pattern sampled per attack run, restricted to the valid normalized input domain; hidden from verifier	Enables meaningful recovered-pattern-similarity evaluation while ensuring the ground-truth trigger represents a valid digital input perturbation
Target label	One target label sampled per attack run; hidden from verifier	The reference repo's demo hardcodes a single target label; this project randomizes it
Primary PMR	0.05 / 0.10	Project's primary operating regime (deliberate deviation — see Section 4.1)
Stress/reproduction PMR	0.45	CrowdGuard paper's own Table IV default, retained here only as a stress test

4.1 Rationale for the PMR Regime Choice

CrowdGuard's paper reports $\text{PMR} = 0.45$ as a principal experimental configuration and also evaluates lower malicious-client ratios. The present project deliberately uses $\text{PMR} = 0.05$ and 0.10 as its primary operating regimes because the subsequent interaction between CrowdGuard's client detection, Wu's historical-update subtraction, and KD repair is a separate experimental question from CrowdGuard's original evaluation. With 20 clients, $\text{PMR} = 0.05$, 0.10 , and 0.45 correspond to 1, 2, and 9 malicious clients, respectively.

The lower-PMR settings provide a larger benign-client margin and allow the project to isolate the behavior of the unlearning-and-recovery pipeline without making the experiment primarily a stress test of the client-detection stage. This is a project-level experimental-design choice, not a claim that CrowdGuard provides stronger guarantees at lower PMR.

$\text{PMR} = 0.45$ is retained as a CrowdGuard-default reproduction/stress condition. The project therefore reports the lower-PMR primary experiments separately from the $\text{PMR} = 0.45$ stress/reproduction experiments.

4.2 Dataset Consistency Fix

$20 \times 2,560 = 51,200$, which exceeds CIFAR-10's 50,000 training samples. The main architecture therefore uses 2,500/client for a disjoint full-training partition. The 2,560/client figure is the configuration used in the CrowdGuard paper; it is not a disjoint partition of CIFAR-10's 50,000 training samples (its Table footnote states this size was chosen for comparability with prior defense papers); it does not appear anywhere in the reference GitHub implementation, which instead splits CIFAR-10 by configurable train/test ratios across just 4 clients. If 2,560/client is retained for exact reproduction of the CrowdGuard paper's configuration, the implementation must explicitly document the resulting overlap/reuse or an alternative sampling scheme.

5. Step 1 — CrowdGuard Detection

CrowdGuard's HLBIM is retained: per-layer hidden activations $\rightarrow$ cosine and Euclidean distances $\rightarrow$ relative distances $\rightarrow$ PCA $\rightarrow$ first principal component $\rightarrow$ significance/iterative pruning $\rightarrow$ binary client votes. The stacked clustering procedure is also retained: two-cluster agglomerative clustering $\rightarrow$ largest cluster $\rightarrow$ DBSCAN ($\text{eps}=0.5$, $\text{min_samples}=1$) $\rightarrow$ representative/frequent vote vector $\rightarrow$ benign indices.

Output: the estimated malicious-client set M . The project retains CrowdGuard's HLBIM and stacked-clustering detection procedure and does not replace its detection/aggregation mechanism with median or trimmed-mean aggregation.

6. Step 2 — Wu Historical-Update Unlearning

Wu et al. formulate historical attacker-update subtraction for a single identified attacker: each round's contribution from that attacker is subtracted back out of the global model, with a small per-round correction term added back in. This project extends the historical-update subtraction procedure to the estimated malicious-client set (M) by subtracting the historical contributions of all clients in (M) under equal-weight FedAvg. This is a project-level multi-client extension of the Wu formulation, rather than an unchanged application of the paper's single-target derivation. For ($M=\{i\}$), the formulation reduces to the single-target subtraction case. For ($|M|>1$), the implementation must explicitly define how the historical contributions are combined and must evaluate the resulting approximation experimentally. The single-attacker case reduces to the original Wu-style subtraction. The paper itself demonstrates experiments with multiple attackers, but the exact multi-client extension used here must be specified and validated, not presented as an unchanged single-attacker method.

In Wu et al.'s original paper, the analogous per-round correction term ϵ_t is never given a closed-form definition. It is introduced conceptually, as the necessary amendment (skew) to the global model at each round t , and the paper's own resolution does not compute it explicitly — Step 2 (knowledge distillation) is used instead to implicitly remedy the accumulated skew $\sum \epsilon_t$, without deriving what ϵ_t equals algebraically. This project's multi-client correction term

is therefore structurally unresolved by design and remedied only implicitly via the KD step (Section 7), not a term that requires further derivation. For the multi-client extension, $\epsilon T, M$ denotes the accumulated residual skew across the rounds through T , i.e. the multi-client analogue of the accumulated $\Sigma \epsilon t$.

Historical state requirement: the server must retain sufficient per-client historical information to reconstruct each identified client's accumulated contribution. The architecture does not require storing every raw tensor indefinitely if an equivalent sufficient statistic can be maintained; the implementation must document exactly what is retained. The server retains the historical update for each client for each round required by the Wu subtraction procedure, preserving client and round association.

The implementation must also preserve the association between each stored historical update and its client identity so that the CrowdGuard-detected set (M) can be mapped to the corresponding historical contributions.

7. Step 3 — Wu Knowledge-Distillation Repair

After historical subtraction, the damaged model θ' is repaired using Wu's KD procedure. The pre-unlearning global model acts as teacher and the unlearned model as student. Clean public/reference data are used to preserve normal predictive behavior. The result is θU .

No client participation is required for the Wu unlearning computation itself. This step is also where the residual round-wise skew ($\epsilon T, M$, Section 6) is remedied implicitly rather than through an explicit algebraic correction.

8. Step 4 — Simplified BAERASER-Inspired Trigger Recovery

The original BAERASER recovery stage uses an entropy-maximization generative mechanism based on a max-entropy staircase approximator (MSA), together with multiple sub-models and a mutual-information estimator, to recover an unconstrained trigger distribution. Its recovery procedure iterates over candidate target labels rather than requiring the target label to be supplied.

This project does not implement that original recovery mechanism. Instead, it uses a simplified BAERASER-inspired recovery procedure tailored to the project's known 6×6 spatial-extent prior.

This project adapts that recovery stage in two ways: (1) it adds a known 6×6 spatial-extent prior while leaving position and RGB pattern unknown, and (2) it deliberately simplifies BAERASER's optimization machinery, dropping MSA and the mutual-information term.

8.1 Why MSA and the Mutual-Information Term Are Dropped

MSA and the mutual-information mechanism are part of BAERASER's original approach to trigger-distribution recovery in the unconstrained setting. This project removes both mechanisms because its recovery search space is restricted to a known 6×6 support. Once the search space is restricted to a small, spatially-localized 6×6 patch, the dimensionality problem that motivated MSA is substantially reduced, and the reduced 6×6 search space makes direct optimization of the trigger tensor a practical project-level alternative. This is an explicit simplification of BAERASER's recovery mechanism, not a claim that MSA is unnecessary in the unconstrained setting the original paper addresses.

8.2 Simplified Recovery Formulation

For each candidate 6×6 location s and candidate target label (y), the verifier defines a fixed spatial mask over that region. For 32×32 CIFAR-10 images, there are $27 \times 27 = 729$ valid top-left locations and 10 candidate labels, giving 7,290 candidate (s, y) pairs, and directly optimizes a continuous learnable ($6 \times 6 \times 3$) trigger tensor. The trigger tensor is optimized using clean verification data to increase the model's probability of the candidate target label when the candidate trigger is applied. The optimized tensor is clipped to the valid normalized input range for realizable-input ASR and visual comparison. The verifier enumerates candidate target labels and candidate 6×6 locations and retains candidates whose batch-level attack success rate exceeds the predefined recovery threshold τASR . The value of τASR must be fixed before evaluation and reported with the recovery protocol.

Unlike the original BAERASER recovery formulation, this project-level procedure uses no generator network, no noise inputs δ or δ' , and no mutual-information term $\eta \cdot H_i(G_i(\delta); \delta')$, since MSA and the MI-based diversity mechanism are not used in this simplified recovery stage (Section 8.1). In BAERASER's original Algorithm 1, δ' was a second independent noise draw used only by the MI estimator H_i ; it plays no role once that estimator is removed.

The location s , target label y , and trigger tensor γ are therefore explicit variables in the recovery search. This 6×6 mask/location/free-tensor formulation is a project simplification of BAERASER's recovery objective; it is not claimed to be the exact mathematical formulation in the BAERASER paper.

8.3 Randomized Ground Truth

To make the position-unknown and pattern-unknown claims experimentally meaningful, each simulated attack instance draws its own random 6×6 position, random continuous RGB pattern within the valid input domain, and random target label; only the 6×6 size is fixed across runs. The reference implementation's demo hardcodes the trigger to a fixed top-left corner and a fixed target label — a valid simplification for a minimal demo, but insufficient for evaluating position or pattern recovery, since recovered-position-error and recovered-pattern-similarity (Section 14) are only meaningful metrics if the ground truth varies across runs.

9. Verification: What Recovery Failure Means

Evidence of residual backdoor: recovered trigger exists, its ASR exceeds τ_{ASR} , and its ASR exceeds the corresponding clean-model recovery baseline by the predefined margin Δ_{baseline} ; clean-task behavior remains separately measurable.

No recoverable trigger found: report "no trigger recovered under the specified protocol," not "backdoor absent."

A recovered trigger that produces high ASR is evidence that residual trigger behavior remains after unlearning. However, failure to recover a trigger is not proof that the model contains no backdoor. Recovery is an empirical search with finite data, optimization, thresholds, and a constrained trigger family.

The final report should therefore distinguish at least: residual-backdoor evidence and no recoverable trigger under the protocol. An inconclusive status should be allowed when recovery fails to converge, computational limits prevent adequate search, or verifier data are insufficient.

10. Independent Client Verification

Remaining clients, meaning clients not in the estimated malicious-client set M , run the recovery and evaluation locally on θ_U . They do not receive the true trigger. Each remaining client computes recovered-trigger ASR and clean accuracy from its own verification data.

The experimenter can later compare the recovered trigger against the hidden ground-truth 6×6 trigger (position, pattern, and label) and measure true-trigger ASR. This is an offline evaluation step and does not leak the ground truth into the verification protocol.

11. Step 5 — Commit–Reveal

Phase 1 — Commit: each remaining client hashes its vote together with the round, a context string, and a private nonce, and publishes only that hash.

Phase 2 — Reveal: remaining clients reveal their vote and nonce; the hash is recomputed and checked against the earlier commitment.

Phase 3 — Independent tally: valid reveals are tallied according to the predefined tally rule.

Security boundary: commit–reveal protects commitment/auditability of votes; it does not make dishonest clients honest and does not protect against a fully malicious server.

Signature requirement: a hash commitment is not a digital signature. Signatures require separate key infrastructure.

12. TEE Decision

CrowdGuard's original system uses TEE/SGX infrastructure on both client and server sides. This project deliberately omits that deployment for reproducibility. This is a scope change, not a claim that commit–reveal replaces TEE security. TEE provides protected execution/isolation and confidentiality of local models; commit–reveal provides vote commitment and reveal integrity only.

13. Experimental Plan and Ablations

Clean-model recovery baseline	Baseline recovery behavior on a model trained without backdoor poisoning
Experiment	Purpose
No defense	Backdoor baseline
CrowdGuard only	Client detection effect
Wu only	Unlearning effect with known/ground-truth malicious set as appropriate
CrowdGuard + Wu	Detection + unlearning
Full proposed pipeline	Detection + Wu + BAERASER-inspired recovery + verification
Known-trigger vs. recovered-trigger verification	Measures the cost/value of hiding the trigger
$\alpha \in \{0.1, 0.5, 1.0\}$	Non-IID robustness
PMR 0.05 / 0.10 (primary); 0.45 (stress)	Malicious-client fraction sensitivity — see Section 4.1 for rationale

14. Evaluation Metrics

Backdoor behavior: pre-unlearning ASR using the ground-truth trigger; post-unlearning ASR using the ground-truth trigger; recovered-trigger ASR; trigger recovery success rate.

Utility: clean test accuracy; macro-F1; change in clean accuracy after unlearning.

Recovery: recovered position error; recovered pattern similarity in normalized continuous pixel space; target-label recovery; convergence/failure rate; runtime.

Verification: verifier agreement; residual-evidence rate; no-recovery rate; inconclusive rate; tally consistency.

Efficiency: CrowdGuard runtime; Wu subtraction runtime; KD runtime; recovery runtime; total defense/verification time; historical-state storage.

Since the recovered trigger is directly optimized, its raw values may fall outside the valid input range. Pattern similarity is therefore computed on the raw continuous tensors in normalized space. A separately clipped version may be evaluated for realizable-input ASR and visual comparison.

15. Claims and Explicit Limitations

1. No universal proof: the system does not establish that every possible backdoor has been removed.

2. Multi-client Wu: the malicious-set subtraction is a project extension and must be mathematically specified and empirically validated. Its correction term is structurally unresolved by design, following Wu et al.'s own treatment, and is remedied only implicitly via knowledge distillation.

3. BAERASER-inspired recovery: BAERASER's max-entropy staircase approximator and mutual-information estimator are deliberately dropped, not retained. The recovery mechanism is replaced with direct optimization of a free $6 \times 6 \times 3$ tensor per (label, position) candidate, justified by the reduced dimensionality of the 6×6 -constrained search space. The 6×6 support/location prior is project-specific and, together with the fixed trigger size, is traced to the CrowdGuard reference implementation's demo code rather than either paper.

4. TEE: omitted intentionally; no SGX isolation/confidentiality claim is made.

5. 6×6 prior: this is additional defender knowledge and must be stated in every experiment. Trigger position, RGB pattern, and target label are randomized per attack instance and hidden from the verifier.

6. Recovery failure: no recovered trigger under the protocol is not equivalent to proof of backdoor absence.

7. Dataset accounting: the main configuration uses 2,500 samples/client so 20 clients exactly cover CIFAR-10's 50,000 training samples. A 2,560/client reproduction (CrowdGuard paper's own convention) must document sample reuse/overlap.

8. PMR regime: $\text{PMR} = 0.05/0.10$ is used as the project's primary regime to provide a larger benign-client margin and to isolate the unlearning-and-recovery pipeline from the CrowdGuard detection stress regime. $\text{PMR} = 0.45$ is retained as a stress/reproduction setting.

9. Client count vs. reference implementation: the 20-client configuration follows the CrowdGuard paper's Table IV, not the public reference implementation, which uses a 4-client demo with a fixed-corner trigger and a fixed target label.

16. Source-to-Project Mapping

Component	Status
CrowdGuard HLBIM + stacked clustering	Retained
CrowdGuard TEE deployment	Intentionally omitted
Wu historical-update subtraction	Primary unlearning; multi-client extension
Wu KD repair	Primary repair; also implicitly remedies unresolved $\epsilon T, M$
BAERASER MSA / mutual-information recovery	Dropped — replaced with direct free-tensor optimization
BAERASER gradient-ascent unlearning	Not used
6×6 trigger size prior	Project-specific; traced to CrowdGuard reference repo demo
Trigger position, RGB pattern, target label	Randomized per instance; verification boundary
20-client / $\text{PMR} = 0.45$ configuration	CrowdGuard paper's Table IV, not the reference repo (4 clients, $\text{PMR} = 0.25$)
Post-unlearning empirical verification	Project contribution/design
Commit-reveal	Project contribution/design
All 20 clients participate every round	Project threat-model boundary

17. Final Architecture Statement

Federated training $\rightarrow$ CrowdGuard detection $\rightarrow$ estimated malicious client set $M \rightarrow$ Wu historical contribution removal $\rightarrow$ Wu KD repair $\rightarrow$ post-unlearning model $\theta_U \rightarrow$ BAERASER-inspired trigger recovery (target label unknown; 6×6 extent known; position and RGB pattern unknown) $\rightarrow$ recovered trigger test $\rightarrow$ residual-backdoor evidence / no recoverable trigger / inconclusive $\rightarrow$ commit-reveal $\rightarrow$ independent tally. Position, RGB pattern, and target label are randomized per attack instance.

Technically defensible framing: the architecture performs post-unlearning, non-invasive empirical verification by attempting to recover residual trigger behavior under a stated 6×6 prior, using a deliberately simplified recovery

mechanism. It does not claim that failure to recover a trigger proves the model is backdoor-free, and it does not claim to reproduce BAERASER's full recovery method.